# MALWARE CLASSIFICATION USING DIFFERENT CNN ARCHITECTURES

**A REPORT SUBMITTED IN**

**PARTIAL FULFILMENT OF THE REQUIREMENTS FOR**

**INTERNSHIP**

**By**

***Savio Tomy***

***Rajagiri School of Engineering and Technology***

**DURING**

***1/5/2024 – 1/7/2024***

**UNDER THE GUIDANCE OF**

***Dr. Kanchan Lata Kashyap***

**INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM**



# CERTIFICATE

This is to certify that the work contained in this internship report entitled "Malware classification using different CNN architectures" submitted by Savio Tomy to the Indian Institute of Information Technology Kottayam as part of the Summer Internship Program has been carried out by him under my supervision and that it has not been submitted elsewhere.



Kottayam – 686635

July 2024

Mentor

Dr. Kanchan Lata Kashyap

Assistant Professor

# DECLARATION

I, **Savio Tomy,** hereby declare that the report entitled "Malware Classification using different CNN architectures" submitted to the Indian Institute of Information Technology Kottayam as part of the Summer Internship Program is an original work carried out by me under the supervision of **Dr. Kanchan Lata Kashyap**. I also certify that all information, data, and references used in this report have been acknowledged properly. I confirm that the content of this report is a true representation of the work completed during my internship.

I also declare that this report has not been submitted, either wholly or in part, to any other university or institution for any other degree or diploma.



Savio Tomy

# ABSTRACT

Malware detection is an important aspect of cybersecurity, requiring efficient models to classify and detect malicious software accurately. This report presents a comparative study of four different machine learning models: a custom Convolutional Neural Network (CNN), VGG16, InceptionV3, and DenseNet121, applied to the task of malware classification. The primary objective is to evaluate and compare the performance of these models in accurately classifying malware into their respective families. In this study, the VGG16 model achieved the highest performance with an average accuracy of 94%, showcasing its effectiveness as a transfer learning model for this task.

The custom CNN achieved an average accuracy of 93% on unseen test data, demonstrating its strong capability in identifying malware. InceptionV3 and DenseNet121 achieved accuracies of 87% and 82% respectively, indicating their potential but also highlighting the challenges of adapting these architectures for malware detection. This comparative analysis provides insights into the strengths and weaknesses of each model, guiding future research and development in the field of malware detection. The findings suggest that while pre-trained models like VGG16 offer significant benefits, custom architectures built specifically for malware detection can also give similar results.

# CONTENTS

1. Introduction 6

2. Literature Review 6

3. Methodology 7

4. Result and Discussion 11

# INTRODUCTION

Malware poses a significant threat to computer systems. They lead to unauthorized access to user data. Malware detection is an important aspect of cybersecurity and is essential for protecting computer systems and sensitive data from malicious software. Efficient antivirus software is necessary to prevent such malicious attacks on computers. We can use various machine learning and deep learning algorithms to perform this task. This study aims to evaluate the effectiveness of different convolutional neural network (CNN) architectures in classifying malware into distinct families.

we compare the performance of a custom-designed Convolutional Neural Network (CNN) against three pre-trained models: VGG16, InceptionV3, and DenseNet121. These models are known for their success in various image recognition tasks and can extract complicated features from an image. The primary objective is determining which model provides the highest accuracy in classifying malware into different families. The aim is to improve the precision and accuracy of such malware classification systems using deep learning.

# LITERATURE REVIEW

During my internship, I reviewed the works of several researchers who employed machine learning algorithms to classify malware. One notable study, **"A Comparative Assessment of Malware Classification using Binary Texture Analysis and Dynamic Analysis"** by Lakshmanan Nataraj, Philip Porras, Jian Zhang, and Vinod Yegneswaran, proposed a static technique for malware classification and compared it with dynamic analysis. This research highlighted the advantages and limitations of both approaches, emphasizing the effectiveness of binary texture analysis.

Another significant contribution, **"Malware Detection Using Deep Learning Algorithms"**, by Mohammed Altaiy, İncilay Yildiz, and Bahadır Ucan compared the performance of three deep learning algorithms: CNN, LSTM, and DNN. This study provided insights into the strengths of each algorithm in handling malware detection tasks and suggested that deep learning algorithms can improve malware detection.

**“Robust Intelligent Malware Detection using Deep Learning”** by R Vinayakumar et al, evaluated classical machine learning algorithms (MLAs) and deep learning architectures based on Static analysis, Dynamic analysis, and image processing techniques for malware detection.

Finally, **"Image-Based Malware Classification Method with the AlexNet Convolutional Neural Network Model"** by Lakshmanan Nataraj et al, proposed a malware classification method based on images and deep learning. This approach visualized malware binary files as

color images, directly generated the required image size, and utilized data augmentation methods to improve the algorithm's performance. The study showed the efficiency of using image-based classification for malware detection.

# METHODOLOGY

### 1. DATASETS

We used two datasets to evaluate the performance of different malware classification models. The first dataset is a malware classification dataset that was used for the Microsoft Malware Classification Challenge. The malware binaries are represented as grayscale images of different resolutions. It is split into a testing set and a training set. There are a total of 8697 files in the training set and 2176 files in the testing set, belonging to 9 different malware families – Gatak(203), Kelihos_ver1(80), Kelihos_ver3(589), Lollipop(496), Obfuscator.ACY(246), Ramnit(307), Simda(9), Tracur(151) and Vundo.

The second dataset used is a combination of the Malimg dataset and the Malevis dataset. Both were blended to form a single dataset that contained all the classes from the Malevis dataset and 5 classes from the Malimg dataset. The dataset has two directories, one each for training and validation. Each set comprises 31 classes. There were 9887 files in total in the training set and 3879 files in the validation set.

### 2. PREPROCESSING

All the images in the datasets were converted into a uniform resolution of 224x224 pixels to match the input requirements of the models. For the training process, data augmentation techniques, including random rotations, shifts, and flips, were used to improve the diversity of the training data. The datasets were organized such that each class had a separate directory in training and testing sets.

### 3. MODEL SELECTION

We compared the performance of four different models to determine which architecture provided the highest accuracy for malware classification. The models selected for this study were:

1. **Custom Convolutional Neural Network (CNN)**

Convolutional Neural Networks (CNNs) are a class of deep learning algorithms specifically designed to process and analyze visual data. CNNs are well suited for image classification tasks due to their ability to recognize patterns in images.

**CNN Architecture:**

**Convolutional layers**

Convolutional layers are the core building blocks of CNNs. They apply a set of filters (kernels) to the input image to produce feature maps that highlight various aspects of the image, such as edges, textures, or patterns. Each filter slides over the image, performing a convolution operation to extract features.

**Pooling Layer (MaxPooling2D)**

This layer performs max-pooling with a 2x2 window to reduce the spatial dimensions of the feature maps, thus reducing the number of parameters and computations required for the network.

**Flatten Layer**

Converts the outputs from the convolutional and pooling layers to a 1D vector, which is used as an input to the fully connected layer.

**Dense Layers**

Dense layers, also known as fully connected layers, perform the final classification based on the features extracted by the convolutional layers. These layers have neurons connected to all activations in the previous layer.

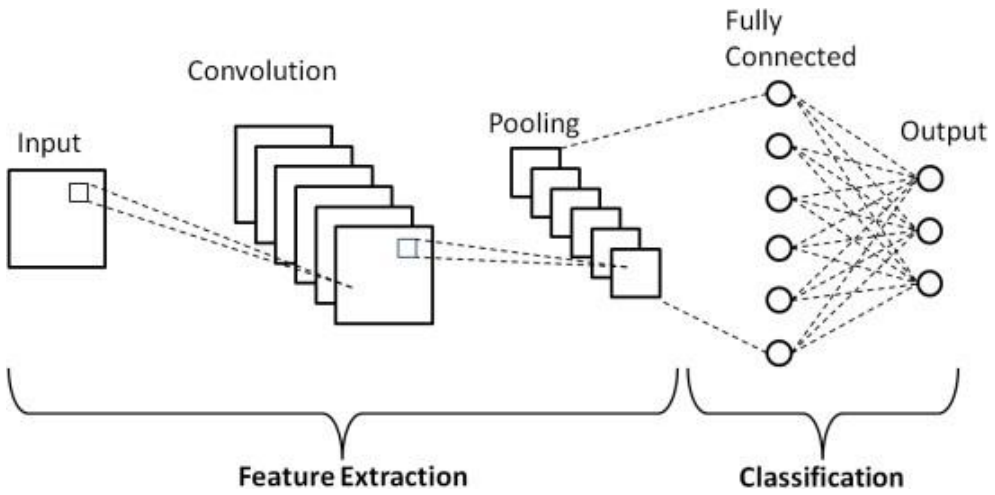



**Figure 1 : CNN**

In our custom model, we used 3 convolutional layers. The first layer had 64 filters, 2nd had 32 and the last convolutional layer had 16. The filter size in each layer is 3x3 and the 'relu' activation function is used. There are also 3 pooling layers with a 2x2 window. The dense layer had 9 neurons corresponding to the number of malware

families. The softmax activation function converts raw scores into probabilities for each class.

## 2. VGG-16

VGG16 is a widely used convolutional neural network (CNN) architecture that was developed by the Visual Geometry Group (VGG) at the University of Oxford. It gained popularity due to its simplicity and effectiveness in various image recognition tasks. VGG16 is widely used for image classification tasks. Pre-trained VGG16 models are used in various domains to use the learned features for new tasks.

VGG16 was originally trained on the ImageNet dataset, which consists of over 1.2 million images across 1000 classes. Due to its success, pre-trained versions of VGG16 are widely used as a starting point for transfer learning. By using weights from the pre-trained model, we can fine-tune the network for specific tasks, which significantly reduces the amount of data and time required for training.

**VGG-16 Architecture**

VGG16 has 16 layers: 13 convolutional layers and 3 fully connected layers. It also uses max-pooling layers to reduce the spatial dimensions. There are five max-pooling layers, each with a 2x2 filter and a stride of 2, following some of the convolutional layers. After the convolutional and max-pooling layers, there are three fully connected layers: the first two have 4096 channels each, and the final layer is a softmax output layer for classification.

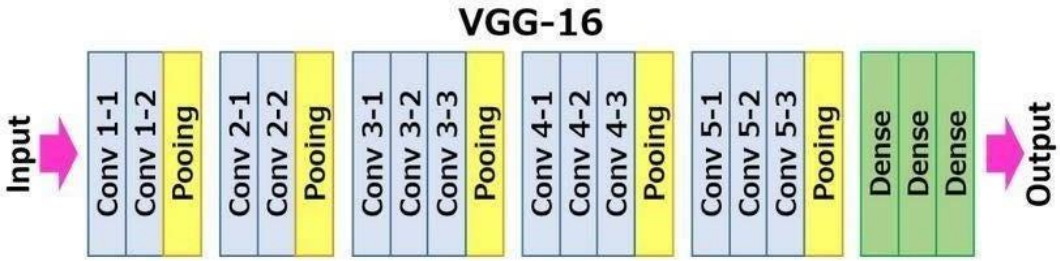



**Figure 2: VGG 16**

## 3. Densenet121

DenseNet121 is a type of convolutional neural network (CNN) architecture that was introduced by the research team at Facebook AI Research (FAIR) and the Department of Information Technology at the University of Tartu. The name "DenseNet" stands for "Densely Connected Convolutional Networks." DenseNet architectures are known for their dense connectivity pattern, where each layer is connected to every other layer in a feed-forward fashion.

**Densenet121 Architecture**

DenseNet121 consists of 121 layers, including convolutional layers, pooling layers, dense blocks, and transition layers. After the last dense block, a pooling layer reduces

the feature maps to a single vector per feature map. A fully connected layer with softmax activation produces the final class probabilities.

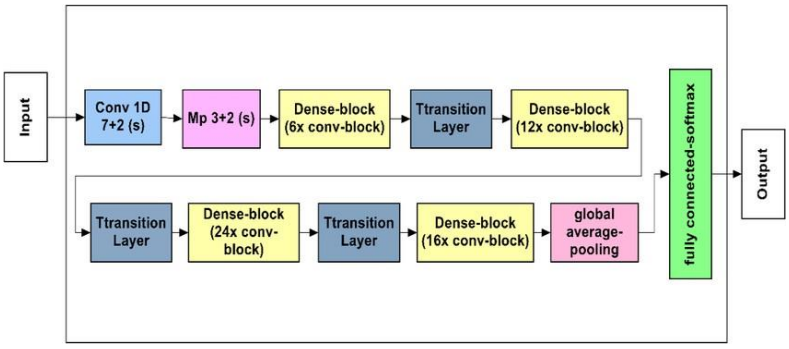



**Figure 3: Densenet 121**

## 4. Inception V3

Inception V3 is a convolutional neural network (CNN) architecture developed by researchers at Google. It is an improved version of the original Inception architecture (also known as GoogLeNet) and builds upon the design principles of its predecessors (Inception V1 and V2). It is a pre-trained convolutional neural network that is 48 layers deep, which is a version of the network already trained on more than a million images from the ImageNet database. Inception V3 is known for its efficiency while maintaining high accuracy in image classification tasks. The architecture is often used for transfer learning.

**Inception V3 Architecture**

Inception V3 is composed of multiple Inception modules, each containing a combination of convolutions, pooling, and concatenation layers. An Inception Module is a building block for convolutional neural networks (CNNs) that allows for the use of multiple filter sizes in a single image block The network has 48 layers, including convolutions, pooling layers, fully connected layers, and softmax output.

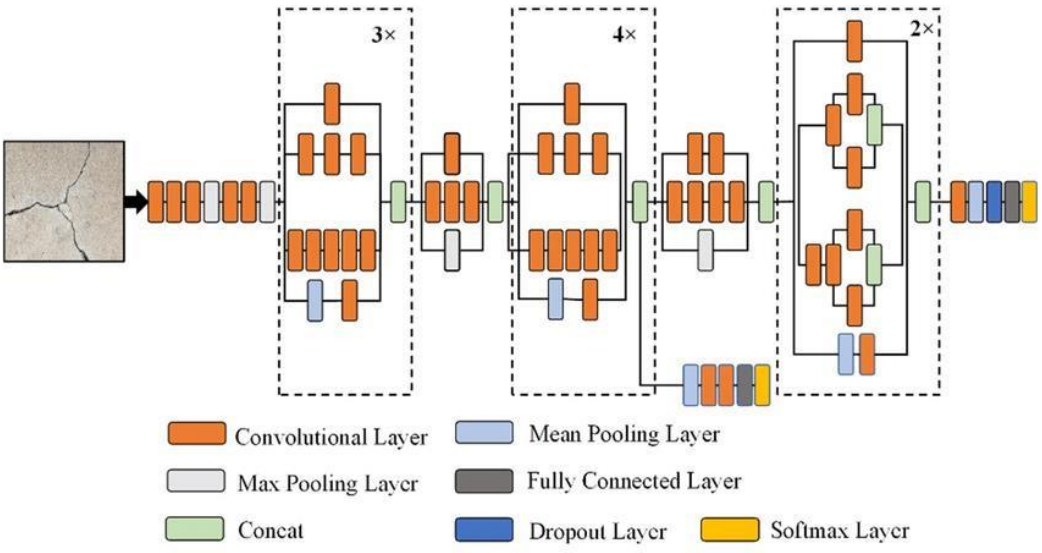


**Figure 4: Inception V3**

## 4. **MODEL TRAINING**

Each model was compiled using the Adam optimizer and the categorical cross-entropy loss function was used for multi-class classification. The models were trained for 10 epochs with a batch size of 32, and the training was monitored using a validation split of 20% of the training data. The training was performed on both datasets separately. The best model was saved based on validation accuracy and the training was stopped when no improvement was observed in the validation loss.

## 5. **MODEL EVALUATION**

After training, the models were evaluated on the test set to assess their performance. The evaluation of the models was conducted on a separate test dataset that was not seen by the models during training. The metrics used for evaluation includes accuracy, precision, recall, F1-score, and confusion matrix.

**Accuracy –** Correct predictions / Total number of predictions

**Precision –** True positives / (True positives + False positives)

**Recall –** True positives / (True positive + False negatives)

**F1 Score –** It is the weighted average mean of Precision and Recall.

**Confusion Matrix -** A confusion matrix is a table that summarizes the performance of a classification model by comparing its predicted labels to the true labels.

## RESULT AND DISCUSSION

In this study, we evaluated the performance of four different convolutional neural network (CNN) architectures for malware classification: a custom CNN, VGG16, InceptionV3, and DenseNet121. Each model was trained on 2 datasets of malware images and evaluated on separate test datasets. The key results are summarized as follows:

### CNN

The custom CNN had a high accuracy among the models, 94% with Dataset 1 and 92% with Dataset 2.

Dataset 1:

```
Found 2176 images belonging to 9 classes.
68/68 [==============================] - 522s 8s/step
                  precision    recall  f1-score   support

           Gatak       0.92      0.96      0.94       203
     Kelihos_ver1       0.95      0.99      0.97        80
     Kelihos_ver3       0.99      0.99      0.99       589
        Lollipop        0.96      0.96      0.96       496
  Obfuscator.ACY       0.87      0.84      0.86       246
          Ramnit       0.91      0.93      0.92       307
           Simda       0.88      0.78      0.82         9
          Tracur       0.91      0.91      0.91       151
           Vundo       0.88      0.87      0.88        95

        accuracy                           0.94      2176
       macro avg       0.92      0.91      0.92      2176
    weighted avg       0.94      0.94      0.94      2176

[[194   0   1   2   3   1   0   0   2]
 [  1  79   0   0   0   0   0   0   0]
 [  2   0 584   1   0   0   0   0   2]
 [  3   0   1 474   5   7   1   3   2]
 [  5   3   0   8 207  13   0   6   4]
 [  2   1   0   5  12 284   0   3   0]
 [  0   0   0   0   0   2   7   0   0]
 [  1   0   0   2   7   3   0 137   1]
 [  2   0   1   2   4   2   0   1  83]]
```

Dataset 2:

```
Found 3879 images belonging to 31 classes.
122/122 [==============================] - 243s 2s/step
              precision    recall  f1-score   support

      Adposhel       0.98      1.00      0.99       144
         Agent       0.91      0.72      0.81       120
       Allaple       0.91      0.93      0.92       128
 Alueron.gen!J       0.96      1.00      0.98        25
     Amonetize       1.00      0.95      0.97       147
        Androm       0.75      0.98      0.85       150
       Autorun       0.98      0.71      0.83       146
     BrowseFox       0.98      0.92      0.95       143
   C2LOP.gen!g       0.96      1.00      0.98        25
Dialplatform.B       1.00      1.00      1.00        25
        Dinwod       0.94      0.99      0.96       149
          Elex       0.97      0.97      0.97       150
        Expiro       0.79      0.79      0.79       151
      Fakerean       1.00      0.99      0.99        75
        Fasong       1.00      1.00      1.00       150
       HackKMS       1.00      0.99      0.99       149
          Hlux       1.00      0.99      1.00       150
      Injector       0.81      0.83      0.82       145
   InstallCore       1.00      0.99      0.99       150
    Lolyda.AA1       1.00      1.00      1.00        60
    Lolyda.AA2       1.00      1.00      1.00        25
     MultiPlug       0.99      0.89      0.93       149
    Neoreklami       0.99      0.97      0.98       150
        Neshta       0.67      0.78      0.72       147
        Regrun       1.00      0.99      1.00       135
        Sality       0.58      0.63      0.60       149
     Snarasite       1.00      1.00      1.00       150
     Stantinko       1.00      0.99      0.99       150
           VBA       1.00      1.00      1.00       150
       VBKrypt       0.92      0.92      0.92       146
        Vilsel       1.00      1.00      1.00       146

      accuracy                           0.92      3879
     macro avg       0.94      0.93      0.93      3879
  weighted avg       0.93      0.92      0.92      3879
```

## VGG 16

VGG16 showed the best performance among all the models with 93% accuracy with Dataset 1 and 95% accuracy with Dataset 2.

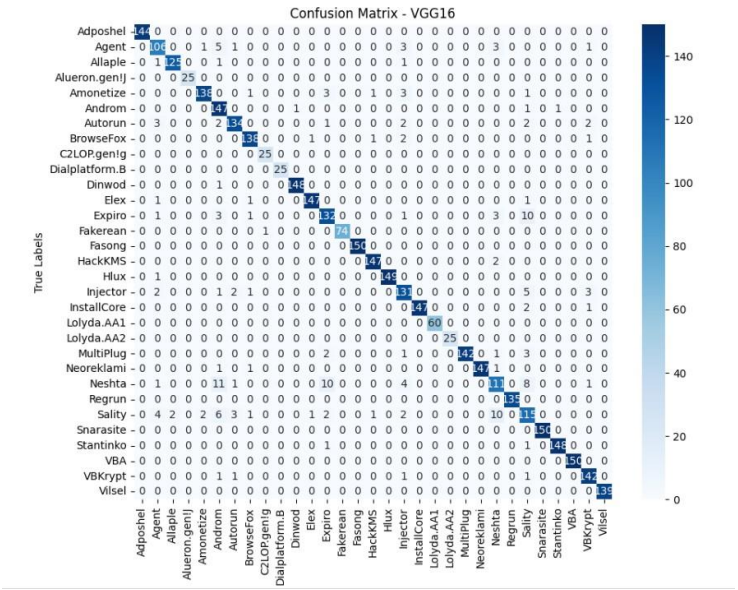
## Dataset 1:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Gatak | 0.89 | 0.95 | 0.92 | 203 |
| Kelihos_ver1 | 1.00 | 0.95 | 0.97 | 80 |
| Kelihos_ver3 | 0.99 | 1.00 | 0.99 | 589 |
| Lollipop | 0.95 | 0.96 | 0.95 | 496 |
| Obfuscator.ACY | 0.97 | 0.83 | 0.89 | 246 |
| Ramnit | 0.90 | 0.93 | 0.91 | 307 |
| Simda | 1.00 | 0.67 | 0.80 | 9 |
| Tracur | 0.76 | 0.91 | 0.83 | 151 |
| Vundo | 0.91 | 0.76 | 0.83 | 95 |
| accuracy | | | 0.93 | 2176 |
| macro avg | 0.93 | 0.88 | 0.90 | 2176 |
| weighted avg | 0.94 | 0.93 | 0.93 | 2176 |




## Dataset 2:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Adposhel | 1.00 | 1.00 | 1.00 | 144 |
| Agent | 0.88 | 0.88 | 0.88 | 120 |
| Allaple | 0.98 | 0.98 | 0.98 | 128 |
| Alueron.gen!J | 1.00 | 1.00 | 1.00 | 25 |
| Amonetize | 0.98 | 0.94 | 0.96 | 147 |
| Androm | 0.82 | 0.98 | 0.89 | 150 |
| Autorun | 0.94 | 0.92 | 0.93 | 146 |
| BrowseFox | 0.96 | 0.97 | 0.96 | 143 |
| C2LOP.gen!g | 0.96 | 1.00 | 0.98 | 25 |
| Dialplatform.B | 1.00 | 1.00 | 1.00 | 25 |
| Dinwod | 0.99 | 0.99 | 0.99 | 149 |
| Elex | 0.99 | 0.98 | 0.98 | 150 |
| Expiro | 0.87 | 0.87 | 0.87 | 151 |
| Fakerean | 1.00 | 0.99 | 0.99 | 75 |
| Fasong | 1.00 | 1.00 | 1.00 | 150 |
| HackKMS | 0.98 | 0.99 | 0.98 | 149 |
| Hlux | 1.00 | 0.99 | 1.00 | 150 |
| Injector | 0.87 | 0.90 | 0.89 | 145 |
| InstallCore | 1.00 | 0.98 | 0.99 | 150 |
| Lolyda.AA1 | 1.00 | 1.00 | 1.00 | 60 |
| Lolyda.AA2 | 1.00 | 1.00 | 1.00 | 25 |
| MultiPlug | 1.00 | 0.95 | 0.98 | 149 |
| Neoreklami | 1.00 | 0.98 | 0.99 | 150 |
| Neshta | 0.85 | 0.76 | 0.80 | 147 |
| Regrun | 1.00 | 1.00 | 1.00 | 135 |
| Sality | 0.77 | 0.77 | 0.77 | 149 |
| Snarasite | 1.00 | 1.00 | 1.00 | 150 |
| Stantinko | 0.99 | 0.99 | 0.99 | 150 |
| VBA | 1.00 | 1.00 | 1.00 | 150 |
| VBKrypt | 0.94 | 0.97 | 0.96 | 146 |
| Vilsel | 1.00 | 1.00 | 1.00 | 139 |
| accuracy | | | 0.95 | 3872 |
| macro avg | 0.96 | 0.96 | 0.96 | 3872 |
| weighted avg | 0.96 | 0.95 | 0.95 | 3872 |

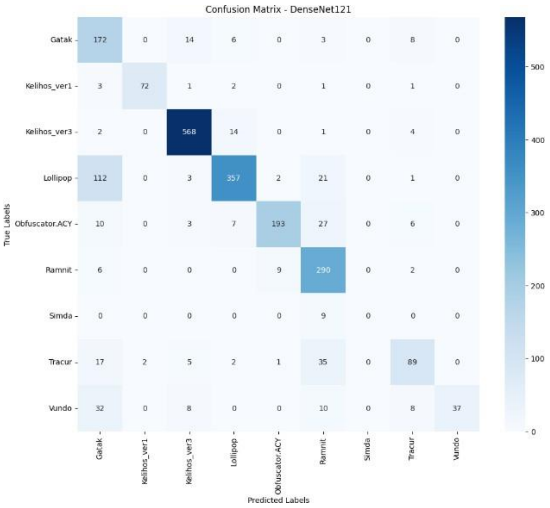


## Densenet121

Densenet121 had the lowest accuracy among all the models, with an accuracy of 82% with both datasets.

Dataset 1

| | | | | |
|---|---|---|---|---|
| Gatak | 0.49 | 0.85 | 0.62 | 203 |
| Kelihos_ver1 | 0.97 | 0.90 | 0.94 | 80 |
| Kelihos_ver3 | 0.94 | 0.96 | 0.95 | 589 |
| Lollipop | 0.92 | 0.72 | 0.81 | 496 |
| Obfuscator.ACY | 0.94 | 0.78 | 0.86 | 246 |
| Ramnit | 0.73 | 0.94 | 0.82 | 307 |
| Simda | 0.00 | 0.00 | 0.00 | 9 |
| Tracur | 0.75 | 0.59 | 0.66 | 151 |
| Vundo | 1.00 | 0.39 | 0.56 | 95 |
| accuracy | | | 0.82 | 2176 |
| macro avg | 0.75 | 0.68 | 0.69 | 2176 |
| weighted avg | 0.85 | 0.82 | 0.82 | 2176 |

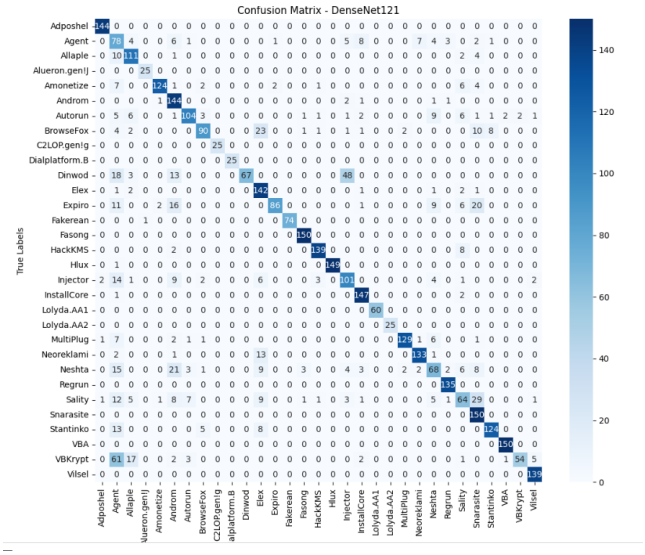


Dataset 2:

```
Test Accuracy: 0.8150826692581177
121/121 ━━━━━━━━━━━━━━━━━━━━ 32s 178ms/step
                precision    recall  f1-score   support

      Adposhel       0.97      1.00      0.99       144
         Agent       0.30      0.65      0.41       120
       Allaple       0.74      0.87      0.80       128
  Alueron.gen!J       0.96      1.00      0.98        25
     Amonetize       0.97      0.84      0.90       147
        Androm       0.63      0.96      0.76       150
       Autorun       0.87      0.71      0.78       146
     BrowseFox       0.87      0.63      0.73       143
    C2LOP.gen!g       1.00      1.00      1.00        25
Dialplatform.B       1.00      1.00      1.00        25
        Dinwod       1.00      0.45      0.62       149
          Elex       0.68      0.95      0.79       150
        Expiro       0.97      0.57      0.72       151
      Fakerean       1.00      0.99      0.99        75
        Fasong       0.96      1.00      0.98       150
       HackKMS       0.95      0.93      0.94       149
          Hlux       1.00      0.99      1.00       150
      Injector       0.61      0.70      0.65       145
   InstallCore       0.88      0.98      0.93       150
    Lolyda.AA1       1.00      1.00      1.00        60
    Lolyda.AA2       1.00      1.00      1.00        25
     MultiPlug       0.97      0.87      0.91       149
    Neoreklami       0.93      0.89      0.91       150
        Neshta       0.63      0.46      0.53       147
        Regrun       0.95      1.00      0.97       135
        Sality       0.62      0.43      0.51       149
     Snarasite       0.65      1.00      0.79       150
     Stantinko       0.93      0.83      0.87       150
           VBA       0.98      1.00      0.99       150
       VBKrypt       0.96      0.37      0.53       146
        Vilsel       0.94      1.00      0.97       139

      accuracy                           0.82      3872
     macro avg       0.87      0.84      0.84      3872
  weighted avg       0.85      0.82      0.81      3872
```

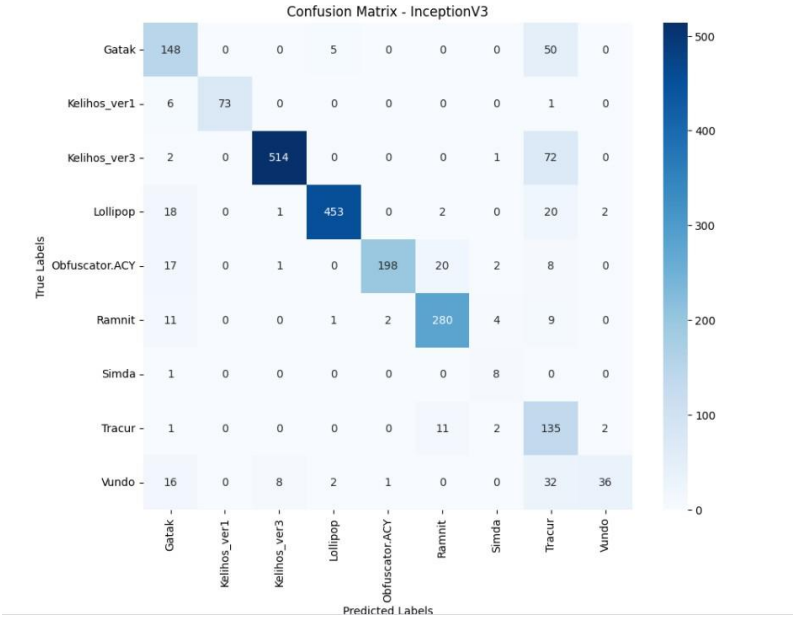



## Inception v3

Inception v3 performed well in malware classification but with an accuracy slightly lower than Custom CNN and VGG 16. Accuracy is 85% for dataset 1 and 89% for dataset 2.

Dataset 1:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Gatak | 0.67 | 0.73 | 0.70 | 203 |
| Kelihos_ver1 | 1.00 | 0.91 | 0.95 | 80 |
| Kelihos_ver3 | 0.98 | 0.87 | 0.92 | 589 |
| Lollipop | 0.98 | 0.91 | 0.95 | 496 |
| Obfuscator.ACY | 0.99 | 0.80 | 0.89 | 246 |
| Ramnit | 0.89 | 0.91 | 0.90 | 307 |
| Simda | 0.47 | 0.89 | 0.62 | 9 |
| Tracur | 0.41 | 0.89 | 0.56 | 151 |
| Vundo | 0.90 | 0.38 | 0.53 | 95 |
| accuracy | | | 0.85 | 2176 |
| macro avg | 0.81 | 0.81 | 0.78 | 2176 |
| weighted avg | 0.90 | 0.85 | 0.86 | 2176 |




Dataset 2:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Adposhel | 0.97 | 1.00 | 0.98 | 144 |
| Agent | 0.67 | 0.66 | 0.66 | 120 |
| Allaple | 0.90 | 0.95 | 0.93 | 128 |
| Alueron.gen!J | 0.96 | 1.00 | 0.98 | 25 |
| Amonetize | 0.99 | 0.84 | 0.91 | 147 |
| Androm | 0.63 | 0.99 | 0.77 | 150 |
| Autorun | 0.75 | 0.86 | 0.80 | 146 |
| BrowseFox | 0.92 | 0.94 | 0.93 | 143 |
| C2LOP.gen!g | 1.00 | 1.00 | 1.00 | 25 |
| Dialplatform.B | 1.00 | 1.00 | 1.00 | 25 |
| Dinwod | 0.95 | 0.97 | 0.96 | 149 |
| Elex | 0.99 | 0.93 | 0.96 | 150 |
| Expiro | 0.65 | 0.83 | 0.73 | 151 |
| Fakerean | 1.00 | 0.99 | 0.99 | 75 |
| Fasong | 0.96 | 1.00 | 0.98 | 150 |
| HackKMS | 0.99 | 0.93 | 0.96 | 149 |
| Hlux | 1.00 | 0.99 | 1.00 | 150 |
| Injector | 0.71 | 0.74 | 0.72 | 145 |
| InstallCore | 0.96 | 0.97 | 0.97 | 150 |
| Lolyda.AA1 | 0.98 | 1.00 | 0.99 | 60 |
| Lolyda.AA2 | 1.00 | 1.00 | 1.00 | 25 |
| MultiPlug | 0.89 | 0.89 | 0.89 | 149 |
| Neoreklami | 0.94 | 0.99 | 0.96 | 150 |
| Neshta | 0.68 | 0.33 | 0.45 | 147 |
| Regrun | 1.00 | 1.00 | 1.00 | 135 |
| Sality | 0.68 | 0.48 | 0.56 | 149 |
| Snarasite | 0.99 | 1.00 | 1.00 | 150 |
| Stantinko | 0.98 | 0.85 | 0.91 | 150 |
| VBA | 1.00 | 1.00 | 1.00 | 150 |
| VBKrypt | 0.90 | 0.84 | 0.87 | 146 |
| Vilsel | 0.99 | 1.00 | 1.00 | 139 |
| accuracy | | | 0.89 | 3872 |
| macro avg | 0.90 | 0.90 | 0.90 | 3872 |
| weighted avg | 0.89 | 0.89 | 0.88 | 3872 |

## CONCLUSION

This study highlights the strengths and limitations of various CNN architectures for malware classification. While VGG16 emerged as the top performer, the custom CNN also demonstrated good performance. In conclusion, both pre-trained models like VGG16 and custom-designed CNNs can be used for malware detection tasks, depending on the goals of classification. Future research should focus on improving these models and exploring their applications in other areas of cybersecurity.

## REFERENCES


1. Lakshmanan Nataraj, Philip Porras, Jian Zhang, and Vinod Yegneswaran **,** "A Comparative Assessment of Malware Classification using Binary Texture Analysis and Dynamic Analysis"
2. Mohammed Altaiy, İncilay Yildiz, and Bahadır Ucan**,** "Malware Detection Using Deep Learning Algorithms"
3. R Vinayakumar et al , “Robust Intelligent Malware Detection using Deep Learning”
4. Lakshmanan Nataraj et al**,** "Image-Based Malware Classification Method with the AlexNet Convolutional Neural Network Model"
5. Figure 1: https://www.researchgate.net/publication/336805909/figure/fig1/AS:817888827023360@1572011300751/Schematic-diagram-of-a-basic-convolutional-neural-network-CNN-architecture-26.ppm

1. Figure 2: https://miro.medium.com/v2/resize:fit:828/format:webp/0*6VP81rFoLWp10FcG
2. Figure 3: https://www.researchgate.net/publication/363850803/figure/fig1/AS:11431281086391947@1664210223016/DenseNet121-architecture.ppm
3. Figure 4: https://www.researchgate.net/publication/349717475/figure/fig5/AS:996933934014464@1614698980419/The-architecture-of-Inception-V3-model.ppm